

Modelo Relacional UML da Aplicação CAMAAR

Sistema de Avaliação de Turmas

Filipe Reis, Gabriel Negreiros, Kelven Silva, Lucas Teixeira
Universidade de Brasília - UnB
Departamento de Ciência da Computação

Novembro de 2025

Resumo

Este documento apresenta o modelo relacional UML da aplicação CAMAAR (Sistema de Avaliação de Turmas), desenvolvida em Ruby on Rails para gerenciamento de avaliações acadêmicas. O sistema permite a gestão completa de turmas, docentes, discentes, templates de formulários, avaliações e respostas, integrando-se com dados do SIGAA.

1 Introdução

A aplicação **CAMAAR** é um sistema desenvolvido em *Ruby on Rails 8.1.1*, utilizando Ruby 3.4.1, projetado para gerenciar avaliações de turmas acadêmicas na Universidade de Brasília. O sistema oferece funcionalidades para criação de templates de formulários, aplicação de avaliações, coleta e análise de respostas, além de integração com dados do SIGAA (Sistema Integrado de Gestão de Atividades Acadêmicas).

Este relatório apresenta o modelo relacional UML completo da aplicação, descrevendo suas entidades principais, atributos, relacionamentos e regras de negócio implementadas.

2 Entidades Principais

2.1 Usuário

Entidade base para todos os usuários do sistema, utilizando herança de tabela única (Single Table Inheritance - STI) para Docente e Docente.

- **usuario**: Identificador único do usuário (chave primária) - string
- **nome**: Nome completo do usuário
- **email**: Email institucional (único)
- **formacao**: Nível de formação (graduando, mestrado, doutorado)
- **ocupacao**: Tipo de ocupação (docente/dicante)
- **password_digest**: Senha criptografada (bcrypt)

- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos: Classe pai de Docente e Docente (relacionamento ISA).

Regras de Negócio:

- Email deve ser único no sistema
- Senha deve ter no mínimo 8 caracteres
- Ocupação determina o perfil de acesso (administrador/participante)

2.2 Docente

Especialização de Usuário representando professores.

- **usuario**: Chave primária e estrangeira (FK para Usuario.usuario)
- **departamento**: Departamento de lotação (ex: "DEPTO CIÊNCIAS DA COMPUTAÇÃO")
- **titulacao**: Nível de titulação (especialização, mestrado, doutorado)

Relacionamentos:

- Um docente pode lecionar várias turmas (1:N com Turma)
- Um docente pode criar várias avaliações (1:N com Avaliacao)
- Um docente pode criar vários templates (1:N com Template)

2.3 Docente

Especialização de Usuário representando alunos.

- **usuario**: Chave primária e estrangeira (FK para Usuario.usuario)
- **matricula**: Matrícula institucional (única)
- **curso**: Curso de graduação (ex: "CIÊNCIA DA COMPUTAÇÃO/CIC")

Relacionamentos:

- Um docente pode estar matriculado em várias turmas (1:N com Matricula)
- Um docente pode responder várias avaliações (1:N com Resposta)

2.4 Materia

Representa disciplinas oferecidas pela instituição.

- **code:** Código da matéria (chave primária) - ex: "CIC0097"
- **name:** Nome da matéria (ex: "BANCOS DE DADOS")
- **created_at:** Data de criação do registro
- **updated_at:** Data da última atualização

Relacionamentos: Uma matéria pode ter várias turmas (1:N com Turma).

Dados Exemplo:

- CIC0097 - BANCOS DE DADOS
- CIC0105 - ENGENHARIA DE SOFTWARE
- CIC0202 - PROGRAMAÇÃO CONCORRENTE

2.5 Turma

Representa uma oferta específica de uma matéria em um semestre.

- **id:** Identificador único (chave primária) - int
- **course_code:** Código da matéria (FK para Materia.code)
- **class_code:** Código da turma (ex: "TA")
- **semester:** Semestre letivo (formato: "YYYY.S", ex: "2021.2")
- **time:** Horário de aula (ex: "35T45")
- **docente_usuario:** Docente responsável (FK para Docente.usuario)
- **created_at:** Data de criação do registro
- **updated_at:** Data da última atualização

Relacionamentos:

- Uma turma pertence a uma matéria (N:1 com Materia)
- Uma turma tem um docente responsável (N:1 com Docente)
- Uma turma pode ter várias matrículas (1:N com Matricula)
- Uma turma pode ter várias avaliações (1:N com Avaliacao)

2.6 Matrícula

Relacionamento entre Docente e Turma, representando a matrícula de um aluno.

- **id**: Identificador único (chave primária) - int
- **dicente_usuario**: Identificador do aluno (FK para Docente.usuario)
- **turma_id**: Identificador da turma (FK para Turma.id)
- **status**: Status da matrícula (ativo/trancado/concluído)
- **enrollment_date**: Data de matrícula
- **final_grade**: Nota final do aluno
- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos:

- Cada matrícula vincula um docente a uma turma
- Um docente pode ter múltiplas matrículas em turmas diferentes
- Uma turma pode ter múltiplas matrículas

Regras de Negócio:

- Um aluno não pode se matricular duas vezes na mesma turma
- Status padrão ao criar matrícula: "ativo"
- Nota final aceita valores de 0.0 a 10.0

2.7 Template

Modelos reutilizáveis de formulários de avaliação criados por docentes.

- **id**: Identificador único (chave primária) - int
- **docente_usuario**: Criador do template (FK para Docente.usuario)
- **name**: Nome do template
- **description**: Descrição detalhada do template
- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos:

- Um template pertence a um docente (N:1 com Docente)
- Um template pode ser usado em várias avaliações (1:N com Avaliacao)

Regras de Negócio:

- Templates podem conter questões dos tipos: numérica (1-5), múltipla escolha, e texto livre
- Questões de múltipla escolha devem ter no mínimo 2 opções
- Templates devem ter ao menos uma questão
- Nome do template deve ser único por docente e semestre

2.8 Avaliacao

Representa uma avaliação específica aplicada em uma turma.

- **id**: Identificador único (chave primária) - int
- **turma_id**: Turma avaliada (FK para Turma.id)
- **docente_usuario**: Docente responsável (FK para Docente.usuario)
- **template_id**: Template utilizado (FK para Template.id, opcional)
- **title**: Título da avaliação
- **description**: Descrição detalhada (text)
- **due_date**: Data limite para respostas
- **max_score**: Pontuação máxima (float)
- **status**: Status da avaliação (rascunho/publicada/fechada)
- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos:

- Uma avaliação pertence a uma turma (N:1 com Turma)
- Uma avaliação pertence a um docente (N:1 com Docente)
- Uma avaliação pode ser baseada em um template (N:0..1 com Template)
- Uma avaliação pode ter várias questões (1:N com Questao)
- Uma avaliação pode ter várias respostas (1:N com Resposta)

Regras de Negócio:

- Avaliações no estado "rascunho" não são visíveis para alunos
- Avaliações "publicadas" são visíveis e podem ser respondidas
- Avaliações "fechadas" não aceitam mais respostas
- Data limite deve ser posterior à data de criação

2.9 Questao

Representa perguntas individuais dentro de uma avaliação.

- **id**: Identificador único (chave primária) - int
- **evaluation_id**: Avaliação associada (FK para Avaliacao.id)
- **order**: Ordem da questão na avaliação (int)
- **question_text**: Texto da pergunta (text)
- **question_type**: Tipo de questão (dissertativa/multipla_escolha/numerica)
- **points**: Pontuação da questão (float)
- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos:

- Cada questão pertence a uma avaliação (N:1 com Avaliacao)
- Questões são ordenadas dentro de cada avaliação

Tipos de Questão:

- **numerica**: Escala de 1 a 5
- **multipla_escolha**: Opções pré-definidas (mínimo 2)
- **dissertativa**: Texto livre

2.10 Resposta

Representa as respostas dos dicentes às avaliações.

- **id**: Identificador único (chave primária) - int
- **evaluation_id**: Avaliação respondida (FK para Avaliacao.id)
- **dicente_usuario**: Aluno que respondeu (FK para Dicente.usuario)
- **submission_date**: Data de submissão
- **score**: Pontuação obtida (float)
- **feedback**: Feedback do docente (text)
- **status**: Status da resposta (pendente/corrigida/atrasada)
- **created_at**: Data de criação do registro
- **updated_at**: Data da última atualização

Relacionamentos:

- Cada resposta pertence a uma avaliação (N:1 com Avaliacao)
- Cada resposta é de um docente (N:1 com Docente)

Regras de Negócio:

- Um aluno pode responder cada avaliação apenas uma vez
- Respostas após a data limite são marcadas como "atrasada"
- Todas as questões obrigatórias devem ser respondidas
- Formulário é bloqueado após submissão bem-sucedida

3 Relacionamentos e Cardinalidades

3.1 Herança (ISA)

- Usuario → Docente (1:1, disjunta, total)
- Usuario → Docente (1:1, disjunta, total)

3.2 Relacionamentos 1:N

- Materia → Turma (1:N)
- Docente → Turma (1:N)
- Docente → Template (1:N)
- Docente → Avaliacao (1:N)
- Turma → Matricula (1:N)
- Turma → Avaliacao (1:N)
- Template → Avaliacao (1:N, opcional)
- Avaliacao → Questao (1:N)
- Avaliacao → Resposta (1:N)
- Docente → Matricula (1:N)
- Docente → Resposta (1:N)

4 Modelo Relacional UML

O modelo relacional UML completo da aplicação **CAMAAR** é apresentado na Figura 1. O diagrama ilustra todas as entidades, atributos, chaves primárias e estrangeiras, além dos relacionamentos e suas cardinalidades.

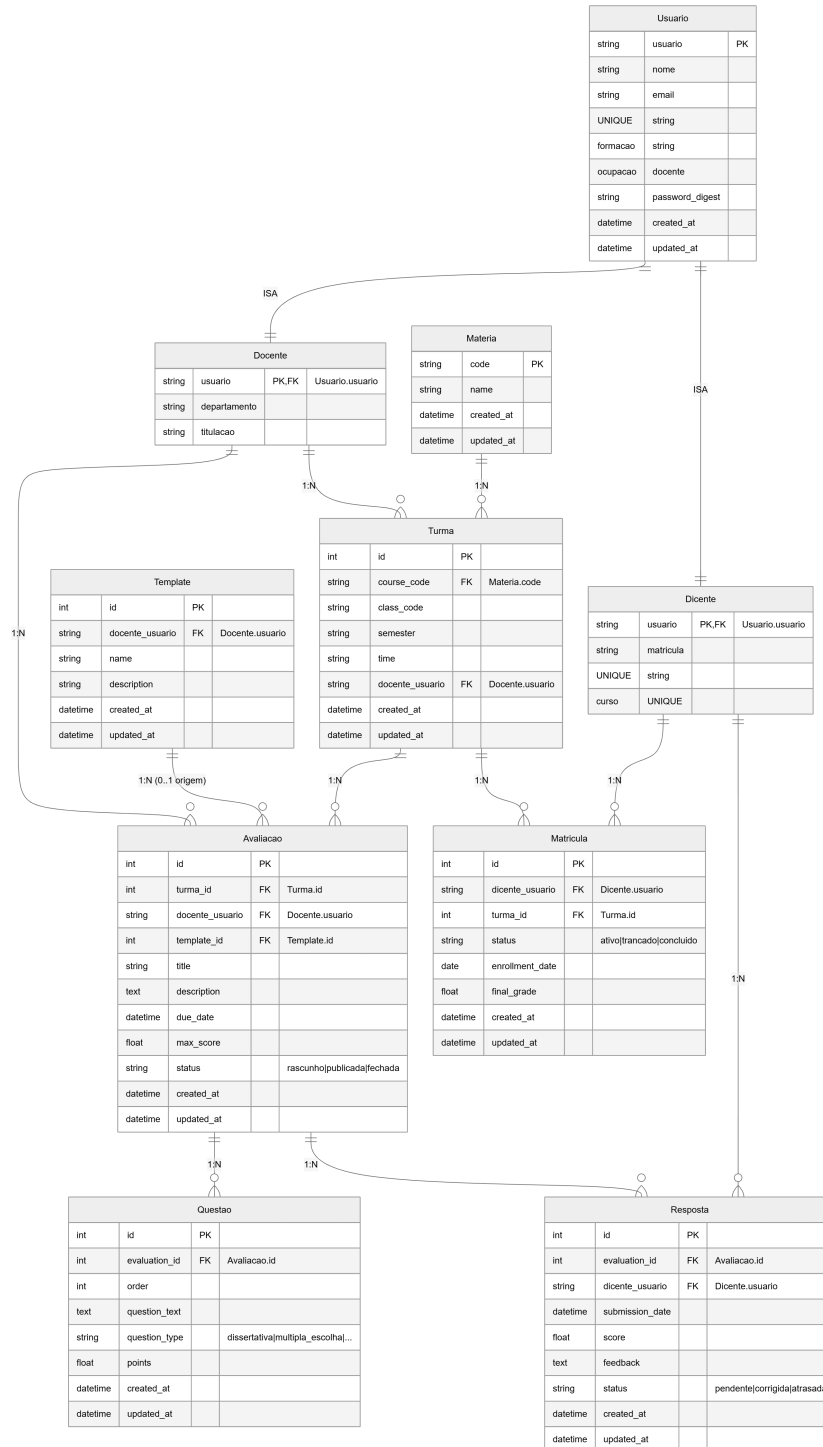


Figura 1: Modelo Relacional UML da aplicação CAMAAR - Sistema de Avaliação de Turmas.

5 Considerações de Implementação

5.1 Tecnologias Utilizadas

- Ruby on Rails 8.1.1: Framework web MVC

- **Ruby 3.4.1:** Linguagem de programação
- **SQLite3:** Banco de dados (desenvolvimento/teste)
- **Cucumber + Capybara:** Framework BDD para testes
- **GitHub Actions:** CI/CD

5.2 Padrões Adotados

Single Table Inheritance (STI): Utilizado para as entidades `Usuario`, `Docente` e `Dicente`, permitindo polimorfismo e reutilização de código.

Soft Deletes: Registros não são deletados fisicamente, apenas marcados como inativos através do campo `status`.

Timestamps: Todos os modelos incluem `created_at` e `updated_at` para auditoria.

Validações: Implementadas no nível de aplicação (`ActiveRecord`) e banco de dados (constraints).

5.3 Segurança

- Senhas armazenadas com `bcrypt` (`has_secure_password`)
- Autenticação baseada em sessões
- Controle de acesso por perfil (`Admin/Participante`)
- Validação de permissões em todas as operações sensíveis

5.4 Integração com SIGAA

O sistema importa dados do SIGAA através de arquivos JSON:

- `class_members.json`: Dados de turmas, docentes e docentes
- `classes.json`: Informações de disciplinas e ofertas

A importação evita duplicação através de validações de unicidade em campos-chave (`usuario`, `matricula`, `email`).

6 Casos de Uso Principais

6.1 UC01: Responder Formulário (#99)

Ator: `Dicente`

Fluxo:

1. `Dicente` visualiza formulários disponíveis para sua turma
2. Sistema valida permissões de acesso
3. `Dicente` preenche questões obrigatórias
4. Sistema valida completude das respostas
5. Sistema registra resposta e bloqueia formulário

6.2 UC02: Cadastrar Usuários do Sistema (#100)

Ator: Administrador

Fluxo:

1. Administrador acessa módulo de cadastro de usuários
2. Sistema solicita dados: nome, email, tipo (docente/dicente)
3. Administrador preenche informações do usuário
4. Sistema valida unicidade de email e matrícula
5. Sistema cria usuário e envia email de ativação

6.3 UC03: Sistema de Login (#104)

Ator: Usuário (Docente/Dicente)

Fluxo:

1. Usuário acessa página de login
2. Sistema solicita credenciais (email/usuário e senha)
3. Usuário fornece credenciais
4. Sistema valida autenticação
5. Sistema redireciona para dashboard apropriado ao perfil

6.4 UC04: Sistema de Definição de Senha (#105)

Ator: Usuário (primeiro acesso)

Fluxo:

1. Usuário acessa sistema pela primeira vez
2. Sistema detecta ausência de senha definida
3. Sistema solicita criação de senha forte
4. Usuário define senha (mínimo 8 caracteres)
5. Sistema valida força da senha e confirma cadastro

6.5 UC05: Criar Template de Formulário (#102)

Ator: Docente

Fluxo:

1. Docente acessa página de gerenciamento de templates
2. Docente define nome e descrição
3. Docente adiciona questões (numéricas, múltipla escolha, texto)
4. Sistema valida regras de negócio
5. Template é salvo e disponibilizado para criação de avaliações

6.6 UC06: Criar Formulário de Avaliação (#103)

Ator: Docente

Fluxo:

1. Docente seleciona turma e template (opcional)
2. Docente define título, descrição e prazo
3. Docente configura questões e pontuação
4. Sistema valida dados e cria avaliação em estado "rascunho"
5. Docente publica avaliação tornando-a visível para alunos

6.7 UC07: Visualizar Formulários Pendentes (#109)

Ator: Docente

Fluxo:

1. Docente acessa dashboard de formulários
2. Sistema lista avaliações disponíveis para suas turmas
3. Sistema destaca prazos próximos e formulários atrasados
4. Docente pode filtrar por turma ou status
5. Docente seleciona formulário para responder

6.8 UC08: Visualizar Resultados dos Formulários (#110)

Ator: Docente

Fluxo:

1. Docente acessa página de resultados
2. Sistema lista avaliações com respostas
3. Docente seleciona avaliação específica
4. Sistema exibe estatísticas consolidadas e gráficos
5. Docente pode exportar relatório

6.9 UC09: Visualizar Templates Criados (#111)

Ator: Docente

Fluxo:

1. Docente acessa biblioteca de templates
2. Sistema lista templates criados pelo docente
3. Docente pode buscar por nome ou filtrar por semestre
4. Sistema exibe preview do template selecionado
5. Docente pode editar, duplicar ou excluir template

6.10 UC10: Edição e Deleção de Templates (#112)

Ator: Docente

Fluxo:

1. Docente seleciona template para editar/excluir
2. Sistema verifica se template está em uso
3. Para edição: docente modifica questões e salva alterações
4. Para exclusão: sistema alerta sobre avaliações vinculadas
5. Sistema confirma operação e atualiza registro

6.11 UC11: Importar Dados do SIGAA (#98, #108)

Ator: Administrador

Fluxo:

1. Administrador acessa página de importação
2. Sistema solicita arquivos JSON
3. Administrador seleciona arquivos
4. Sistema valida formato e integridade
5. Sistema importa dados evitando duplicatas
6. Sistema exibe relatório de importação

6.12 UC12: Gerar Relatório do Administrador (#101)

Ator: Administrador

Fluxo:

1. Administrador acessa módulo de relatórios
2. Sistema oferece opções de filtros (período, turma, docente)
3. Administrador seleciona parâmetros desejados
4. Sistema processa dados e gera visualizações
5. Administrador pode exportar em PDF, Excel ou CSV

7 Métricas do Projeto

7.1 Sprint 1 - Velocity

ID	Funcionalidade	Pontos
#98	Importar dados do SIGAA (base)	5
#99	Responder formulário	5
#100	Cadastrar usuários do sistema	3
#101	Gerar relatório do administrador	5
#102	Criar template de formulário	8
#103	Criar formulário de avaliação	5
#104	Sistema de Login	5
#105	Sistema de definição de senha	3
#108	Atualizar base de dados com dados do SIGAA	5
#109	Visualização de formulários para responder	3
#110	Visualização de resultados dos formulários	8
#111	Visualização dos templates criados	2
#112	Edição e delegação de templates	5
Total (Velocity):		61

7.2 Funcionalidades Bônus (Não Incluídas no Velocity)

- **#106** - Sistema de gerenciamento por departamento (8 pontos)
- **#107** - Redefinição de senha (3 pontos)
- **#113** - Criação de formulário para docentes ou discentes (5 pontos)

8 Considerações Finais

O modelo apresentado reflete uma arquitetura robusta e escalável para o sistema **CA-MAAR**. A estrutura permite:

- **Flexibilidade:** Templates reutilizáveis e tipos variados de questões
- **Escalabilidade:** Suporte a múltiplas turmas, matérias e semestres
- **Auditoria:** Timestamps e status em todas as entidades críticas
- **Integridade:** Relacionamentos bem definidos com constraints
- **Extensibilidade:** Estrutura preparada para novas funcionalidades

O sistema está em conformidade com as práticas recomendadas de desenvolvimento Rails e segue princípios SOLID, facilitando manutenção e evolução contínua.

Referências

- Ruby on Rails Guides: <https://guides.rubyonrails.org/>
- Cucumber Documentation: <https://cucumber.io/docs/>
- GitHub Repository: <https://github.com/EngSwCIC/CAMAAR>
- Figma Mockups: <https://www.figma.com/design/5GVzfaJSBbcXmGvuvAi7WF/>